# 鲜境 · 设计文档

*模块化架构与关键一致性设计*

独立品牌 · 仿盒马生鲜选购体验 · 实际持久化业务闭环

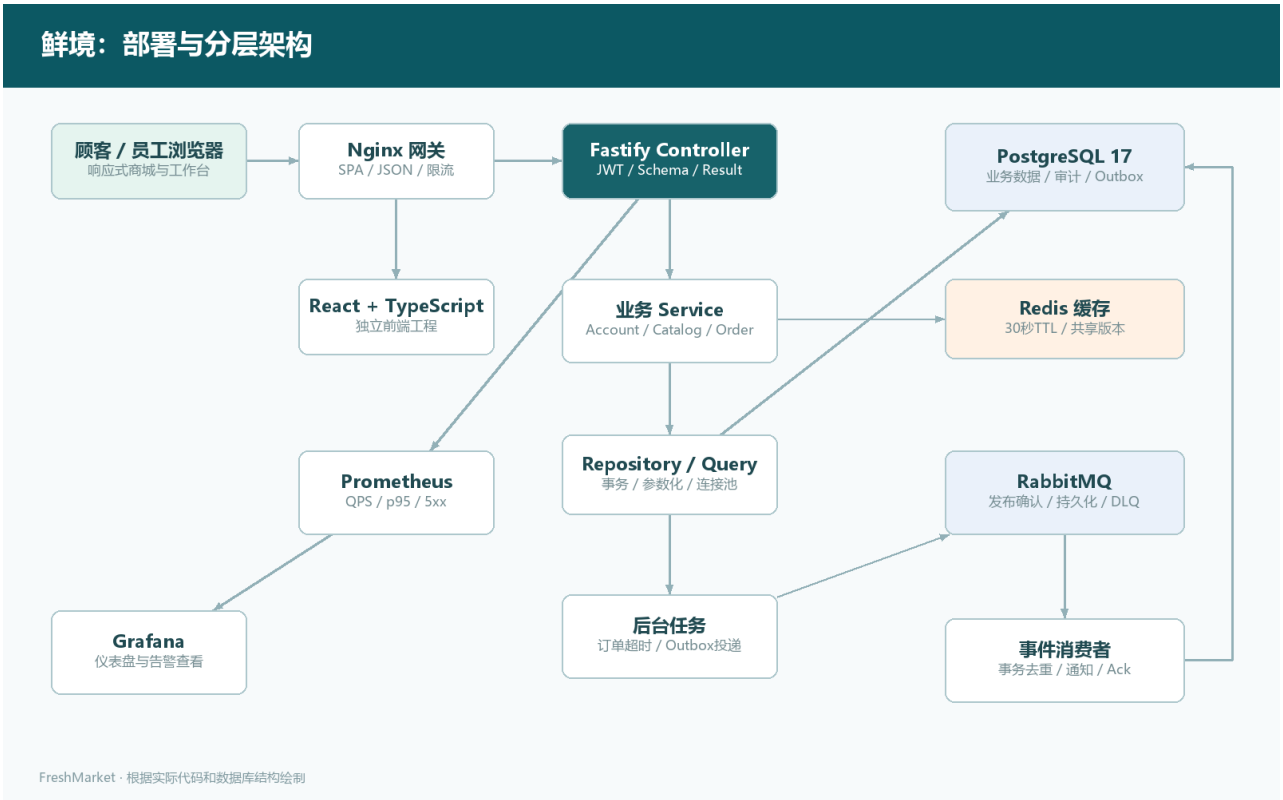
版本 1.0 | 编制日期 2026-10-06

依据实验任务书与教材第九章；记录本次实际实现及测试范围。

提交人姓名 / 学号 / 专业：_____

# 1. 总体架构

采用前后端分离、模块化单体架构。React 工程只调用 JSON 接口；Fastify 控制器负责鉴权/参数/响应，Service 负责业务不变量，Repository 与统一 Query 接口负责数据库访问。通过单数据库事务保持购物核心一致性，用 Outbox 处理业务与消息队列之间的最终一致性。




部署与模块架构 · 完整原图：diagrams/architecture.png

# 2. 技术选型

| 层 | 采用 | 原因 | 替代方案/风险 |
| --- | --- | --- | --- |
| 开发语言 | TypeScript；Node24 | 当前环境已有 Node；前后端共享类型表达，便于复现 | Java/Spring Boot 成熟但需 JDK/Maven；TS 仍需要运行时校验 |
| 前端 | React、Vite、React Router | 组件化、独立构建、响应式商城与管理工作台 | Vue 可替代；大型应用需继续拆包 |

| 层 | 采用 | 原因 | 替代方案/风险 |
| --- | --- | --- | --- |
| API | Fastify5 | JSON Schema、JWT、OpenAPI、注入测试与结构化日志 | Nest/Spring 提供更强约束；本项目显式分层 |
| 主数据库 | PostgreSQL17 | 事务、行锁、条件更新、索引、唯一约束、连接池 | MySQL 可替代；没有实现跨区域多主 |
| 本地数据库 | Node 内置 SQLite | 无外部服务可立即运行；WAL、持久化 | 仅开发/演示，不作为生产高并发部署 |
| 缓存 | Redis7.4 | 商品查询 30 秒 TTL+共享版本失效 | 不缓存付款或库存决策；故障回源数据库 |
| 队列 | RabbitMQ4.3.6 | 持久化队列/消息、发布确认、消费 ack、DLQ | Kafka 适合大吞吐事件流，本规模无需额外复杂度 |
| 网关 | Nginx1.28 | 静态资源、SPA 路由、JSON 代理、登录限流 | 教学部署仅本机 HTTP；公网需 TLS 终止 |
| 监控 | Prometheus3.13.4、Grafana13.2.3 | RT/QPS/5xx/进程指标、可配置告警与仪表盘 | 不包含短信/邮件告警通道及独立 Alertmanager |
| 工程化 | npm workspaces、GitHub Actions、Docker Compose | 锁定依赖、可复现构建、测试、角色化部署 | CI 文件已交付，GitHub 上运行结果单独验收 |

教材使用 Java/若依/Vue/MySQL，实验任务书将技术栈列为建议，允许按团队与环境选型。本项目保留教材的首页、购物车、支付、订单及权限分层思路，采用已有环境易复现的 TS 模块化架构。当前没有微服务拆分，因此不部署 Nacos/Consul；配置由环境变量承担，模块间为进程内调用。详见 docs/adr.md。

依赖精确版本由 package-lock.json 锁定，使用 npm ci 安装；可查询 docs/evidence/dependencies.txt。容器版本来自对应项目发布文档，生产可进一步锁定镜像 digest。

## 3. 模块与分层

| 模块 | Controller | Service | Repository/数据库职责 |
| --- | --- | --- | --- |

| 模块 | Controller | Service | Repository/数据库职责 |
| --- | --- | --- | --- |
| Accounts | 注册/登录/购物车/地址/消息 | 密码校验、购物车数量、地址默认、评价资格 | 通过统一 Query 接口参数化读写用户、购物车与地址 |
| Catalog | 分类/列表/详情 | 缓存、动态筛选、商品维护、库存调整 | Query 接口读写产品/分类；唯一排序枚举 |
| Orders | 创建/查询/付款/取消/收货/退款/评价 | 状态机、事务、幂等、快照、售后/结算约束 | OrderRepository 统一订单行锁、审计、Outbox 与回库存 |
| Admin | 经营数据、商品、履约、售后、财务、角色、审计 | 调用领域 Service，限制 RBAC | 聚合查询与脱敏；权限变更同时审计 |
| Core | 全局异常/Result/JWT/指标/Schema | Runtime 负责缓存和消息，定时任务调度 | PostgreSQL 连接池与 SQLite 事务串行互斥 |

Accounts/Catalog 的数据访问集中于统一数据库适配器，不使用生成的 Mapper 层；OrderRepository 单独封装跨操作复用的一致性逻辑。后台聚合只读取当前数据库的实际数据，零订单时显示真实空状态。

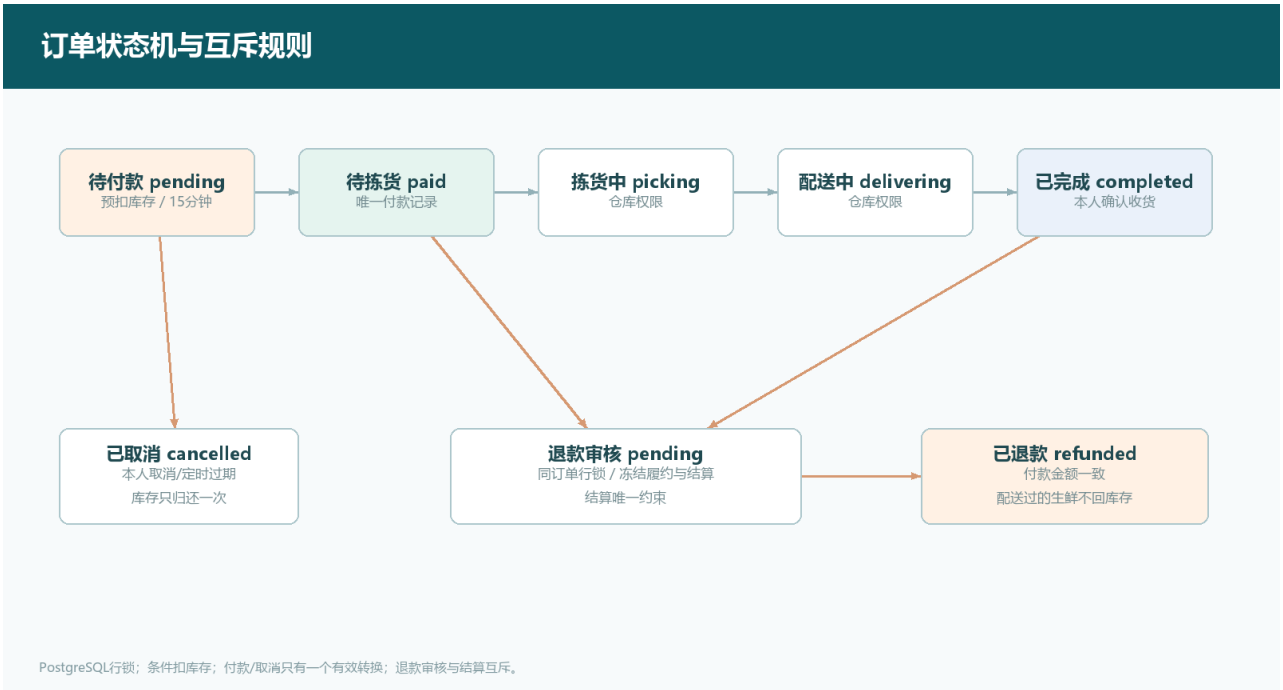
# 4. 一致性设计

## 4.1 下单事务

1. PostgreSQL 先锁定当前用户行，串行化同一用户的购物车修改和下单，解决首笔幂等插入竞争。
2. 查询 (user_id,idempotency_key)；存在则比较 SHA256 请求摘要，摘要相同返回原订单，不再扣库存。
3. 验证地址归属、购物车商品状态和数量，从服务端读取价格。
4. 商品按 id 排序执行条件扣减：UPDATE products SET stock=stock-quantity WHERE stock>=quantity AND active=1。
5. 任一商品更新行数不是 1 则抛 409，整个事务回滚；所有商品库存、购物车、订单和 Outbox 回到提交前状态。
6. 写订单金额/地址/商品快照，清理本人的购物车，写审计及 Outbox，同一事务提交。

不使用“先查库存再无条件更新”的方式，也不把 Redis 库存作为账务依据。金额与原价以整数分保存；下单后修改售价和地址不影响已创建订单快照。

## 4.2 订单状态机




状态与并发规则 · 完整原图：diagrams/state-machine.png

待付款→待拣货→拣货中→配送中→已完成。待付款可取消；付款/取消在同一订单行锁下争抢，只有一个有效转换。15 分钟过期订单由 2 秒轮询任务处理，锁内再次验证是否过期且仍待付款，防止刚付款就被取消。

## 4.3 沙箱支付

只能本人访问沙箱支付接口；服务端从订单快照读取应付金额，客户端不能提交修改后的金额。订单行锁与 payments.order_id UNIQUE 限制单次记账；重试返回原业务结果，销售数量只增加一次。支付并非真实第三方回调，不把其当作具备支付签名验证的生产渠道。

## 4.4 退款与结算

申请退款和财务结算均锁定订单行；售后 pending 时禁止履约/结算，已结算订单在本版本进入人工客服处理。退款审核成功更新订单与沙箱支付状态，扣回销售数量；未发出的商品归还可售库存，已配送/完成的生鲜不回库存。

refunds.order_id 与 settlements.order_id 均为唯一约束。结算只处理 completed 且无 pending/approved 售后的订单；重复批次跳过已经结算订单，避免双重账务。当前没有商家多主体、佣金、税费和实际资金清分。

### 4.5 Outbox 与队列

业务提交时在同事务写 Outbox，不在业务事务里调用远程 RabbitMQ。后台任务每次最多领取 20 条 pending 事件；PostgreSQL 使用 FOR UPDATE SKIP LOCKED；领取状态 processing，60 秒租约超时会重新入队。事件通过 confirm channel 发出，收到发布确认后才标记 sent。

消费者在数据库事务中先写 processed_events.id，冲突说明重复消息，直接 ack；新事件和顾客消息同事务提交后才 ack。消息处理异常进入 fresh.dead 死信队列，供人工检查和重放。声明了 AMQP 配置但队列断开时保留 Outbox，并每 30 秒重连，不切换到另一套传输造成乱序。发布确认与数据库标记之间崩溃可能重复发布，消费端去重保证业务效果一次。

## 5. 安全设计

- JWT 只包含主体 id，有效期 2 小时；每次请求读数据库角色，客户端不能自报角色。成员注册 Schema 拒绝额外 role 字段。
- scrypt + 随机 salt + timingSafeEqual，不保存明文密码；日志与审计不包含密码或 JWT。
- 订单/地址/评价使用用户归属校验；越权订单返回 404，避免泄露对象是否存在；职能越权返回 403。
- SQL 使用绑定参数；动态排序来自固定映射。正文以 React 文本节点显示，不使用 dangerouslySetInnerHTML 渲染顾客输入。
- Bearer 令牌保存在当前浏览器 sessionStorage；采用 Authorization 头，不自动携带会话 Cookie。没有实现 HttpOnly Cookie/refresh token，不能将该实验认证包装为完备生产认证体系。
- API 统一输入校验、64KiB 请求体上限、全局限流 300 次/分钟、登录 10 次/分钟、注册 8 次/分钟；Nginx 有额外登录限制。应用限流为单实例内存计数，多个副本需共享计数存储或统一边缘限流。
- 客服/仓库查看订单手机号脱敏；审计记录操作者、对象、事件、时间与 traceId。订单地址仍受权限控制，未接入业务级字段加密。
- 教学 Compose 只绑定 127.0.0.1，指标接口在 Nginx 阻止外部访问，数据库与缓存无公开端口。公网部署按运维文档配置 HTTPS。

## 6. 缓存、性能与可观测性

商品列表缓存 30 秒，key 包含 Redis 共享版本、查询条件与是否后台；订单/商品/库存变更使版本递增。Redis 失败回源数据库，不影响核心业务提交。缓存中的库存可能短暂落后；实际加购与下单以数据库校验为准。商品详情不缓存，及时读取评价与库存。

PostgreSQL 池最大 15 连接、连接等待 5 秒、空闲 30 秒；订单用户/时间索引、过期状态索引、Outbox 状态索引支持常见操作。SQLite 采用 WAL 与互斥队列，读写不会看到同连接的未提交状态；该模式不作为横向扩展方案。

Fastify 结构化 JSON 日志每个请求分配 UUID traceId，同时出现在响应体和 x-trace-id 头。Prometheus 导出请求计数、响应直方图、内存和事件循环指标，route 标签使用路由模板，避免订单 id 造成高基数。Grafana 展示 QPS、p95、错误率等；Prometheus 已提供服务掉线、5xx 超过 1%、p95 超过 500ms 的告警规则，未接外部通知渠道。

## 7. 工程与风险

单元/接口集成使用 Node test 与 Fastify inject；端到端使用 Playwright；类型检查与生产构建在 GitHub Actions 串行门禁；PostgreSQL 测试在独立 schema 隔离。锁文件、配置示例、SQL、Docker 镜像构建、监控配置、备份/恢复/回滚脚本随源码交付。

本版本主要边界：没有真实支付/物流、部分退款、多商户、优惠券、实时地理围栏、商品批次溯源、自动退款资金渠道、生产高可用和灾备演练。没有为了技术清单堆入 Kubernetes、服务发现或跨数据库 Saga/TCC。横向扩展前还需验收集群限流、种子数据并发初始化和队列消费者的故障注入。

## 8. 主要官方参考

- Node24 SQLite API（https://nodejs.org/download/release/latest-v24.x/docs/api/sqlite.html）
- Fastify 参数校验（https://fastify.dev/docs/latest/Reference/Validation-and-Serialization/）
- PostgreSQL 版本维护（https://www.postgresql.org/support/versioning/）
- RabbitMQ 版本生命周期（https://www.rabbitmq.com/release-information）
- Prometheus 下载与版本（https://prometheus.io/download/）
- Grafana 官方下载（https://grafana.com/grafana/download）

教材及实验原始文件保留在工作区，未将教材扫描页复制进公开交付包。